Interactive session
It’s time to go hands-on!



©EDB 2025 – ALL RIGHTS RESERVED.

Hand-on documentation



Download this presentation

https://tinyurl.com/hands-on-doc

©EDB 2024 — ALL RIGHTS RESERVED.

Environment

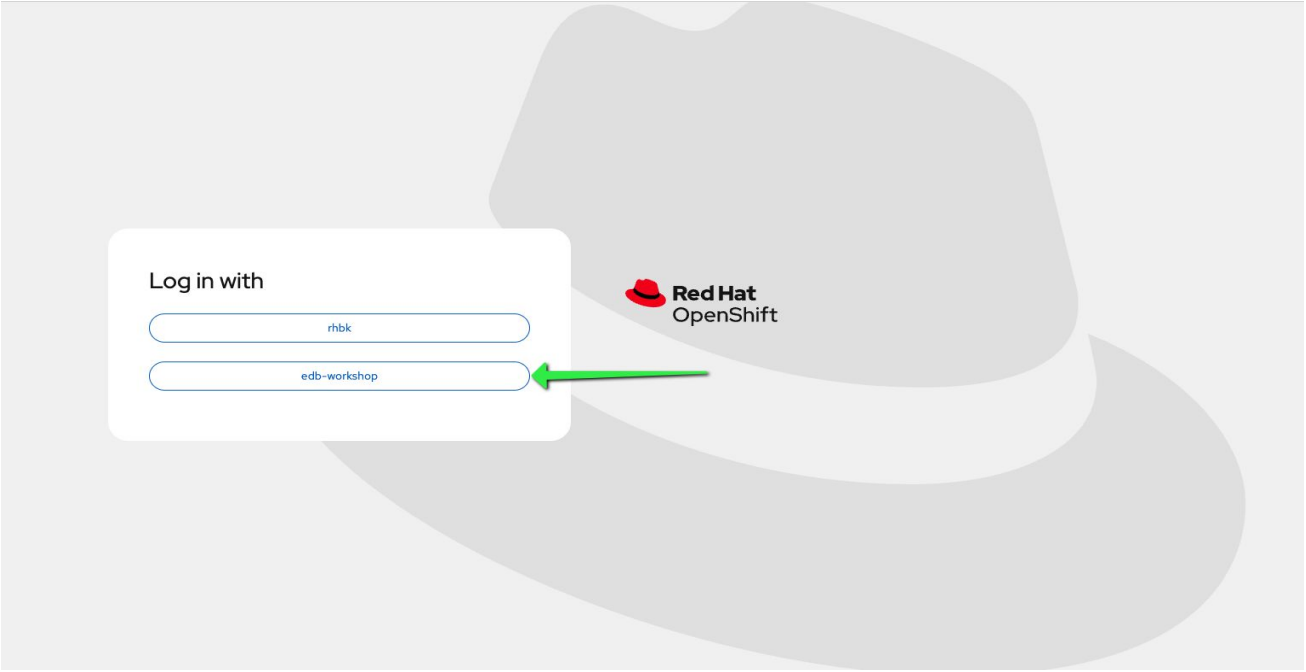
OpenShift Console	https://console-openshift-console.apps.ocp.2lt49.sandbox2071.opentlc.com/
OpenShift User name	user2..user32
OpenShift Password	edb-workshop
Devspaces url	https://devspaces.apps.ocp.2lt49.sandbox2071.opentlc.com
Minio UI	https://minio-ui-minio.apps.ocp.2lt49.sandbox2071.opentlc.com
Minio API	https://minio-api-minio.apps.ocp.2lt49.sandbox2071.opentlc.com
Minio User	minio
Minio password	edb-workshop

©EDB 2025 — ALL RIGHTS RESERVED.

Red Hat OpenShift Login

Got to the OpenShift Console URL, choose edb-workshop:

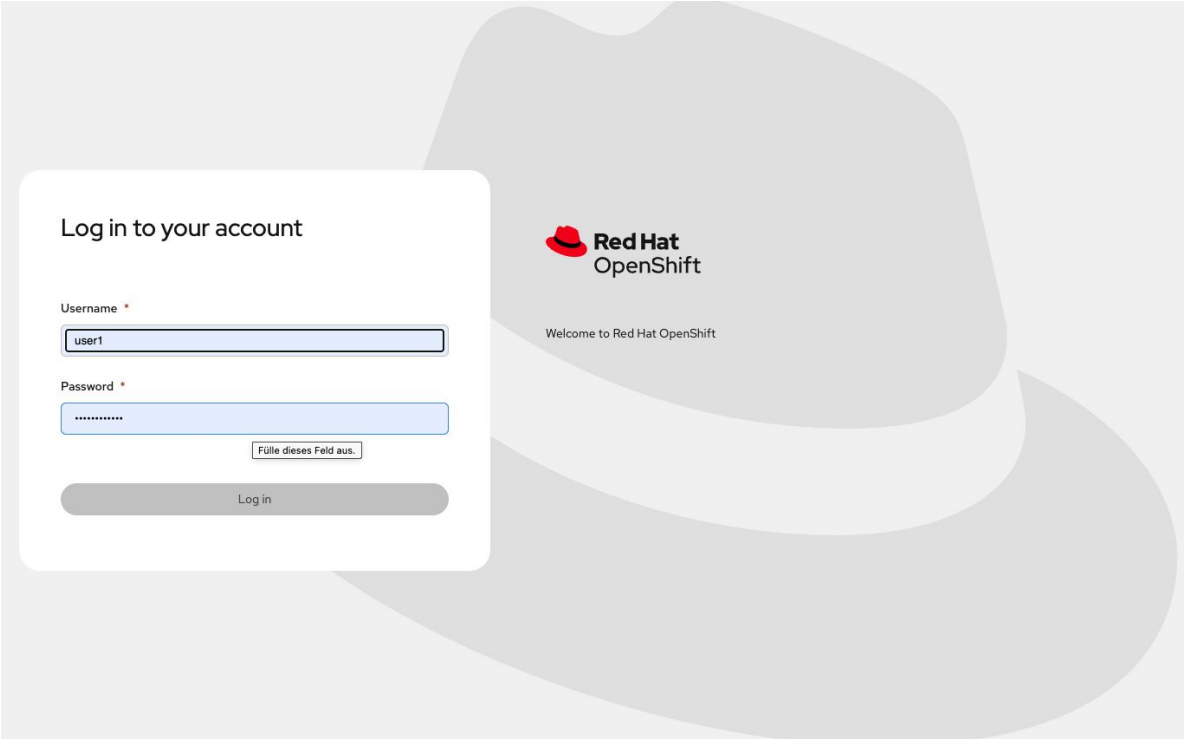
https://console-openshift-console.apps.ocp.2lt49.sandbox2071.opentlc.com/



©EDB 2025 — ALL RIGHTS RESERVED.

Red Hat OpenShift Login

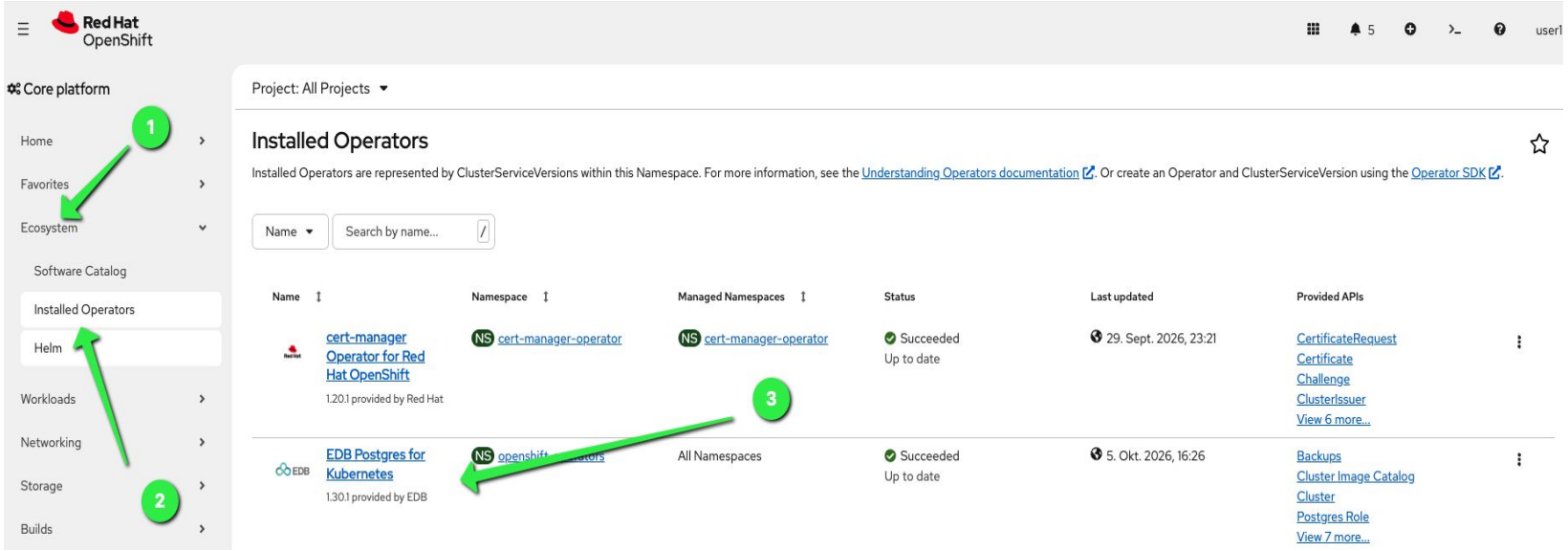
Enter your username and password (edb-workshop):



©EDB 2025 — ALL RIGHTS RESERVED.

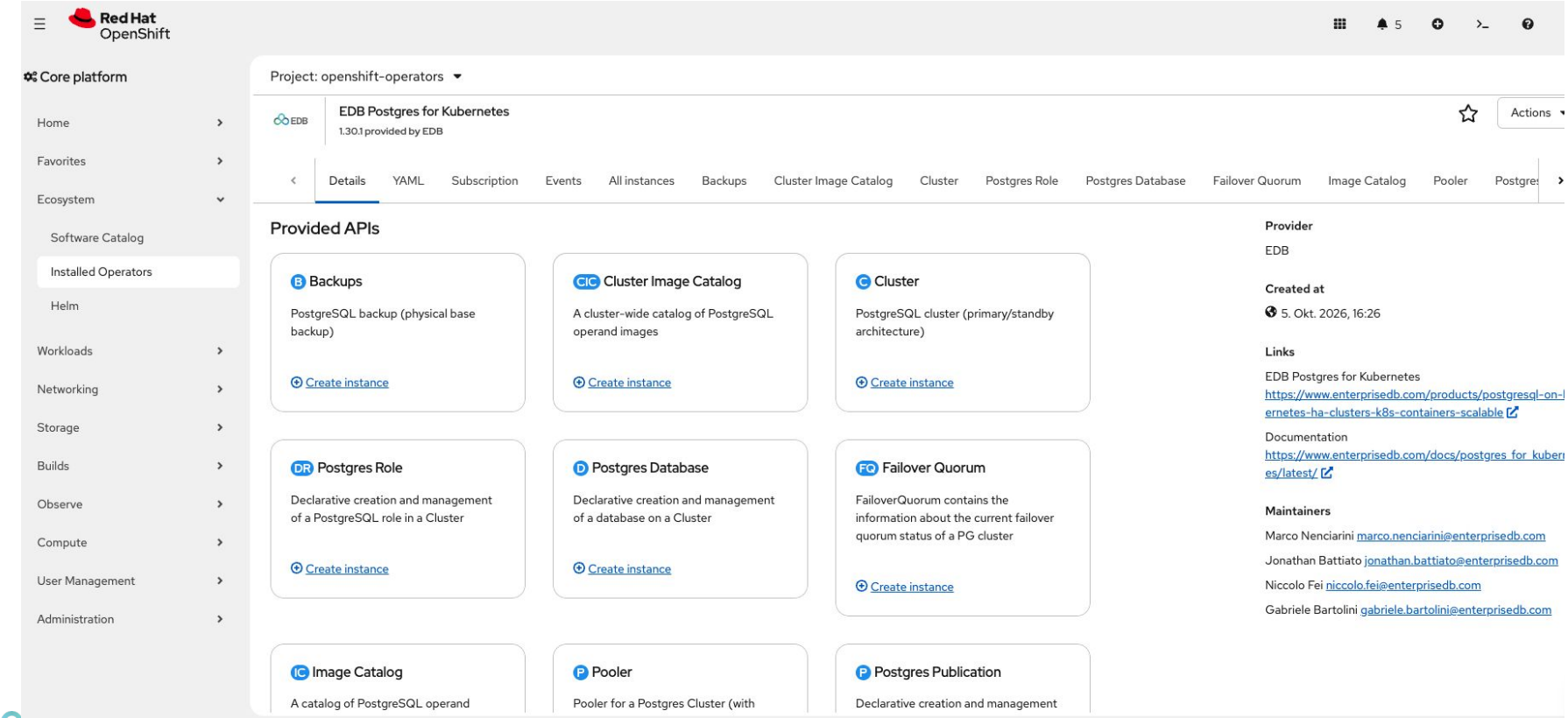
Check installed Operator

Navigate:



©EDB 2025 — ALL RIGHTS RESERVED.

Red Hat OpenShift operator

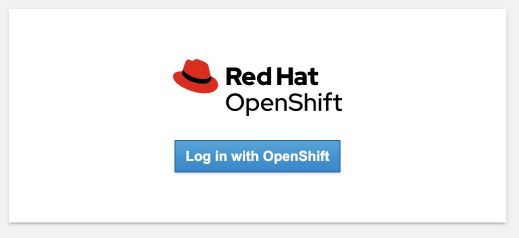


Call to action: Open the DevSpace

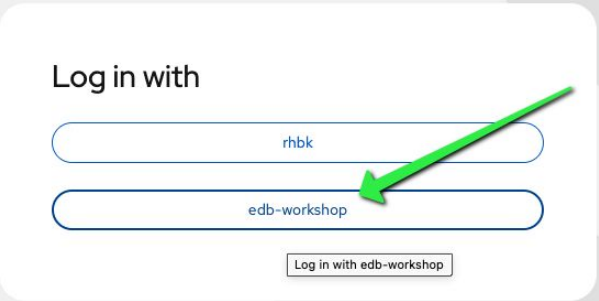
Open the following URL in your browser:

https://devspaces.apps.ocp.2lt49.sandbox2071.opentlc.com

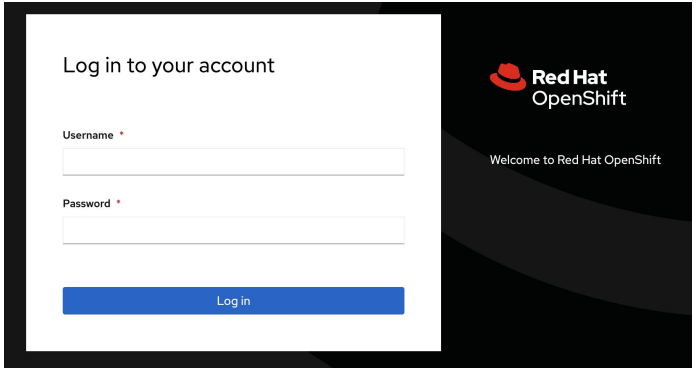
1.



2.



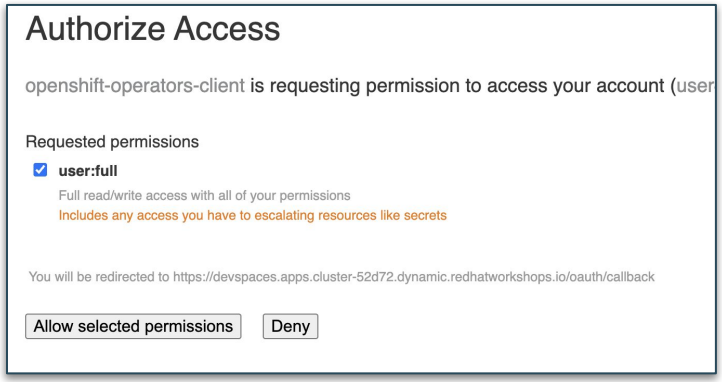
3.



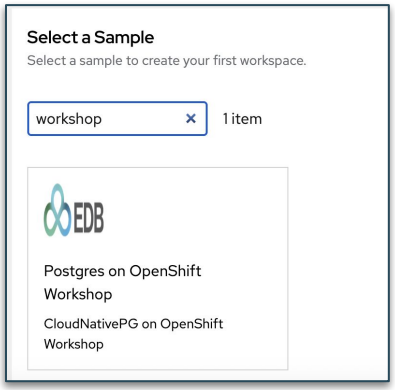
Username and password provided to you

©EDB 2025 — ALL RIGHTS RESERVED.

Call to action: Open the DevSpace



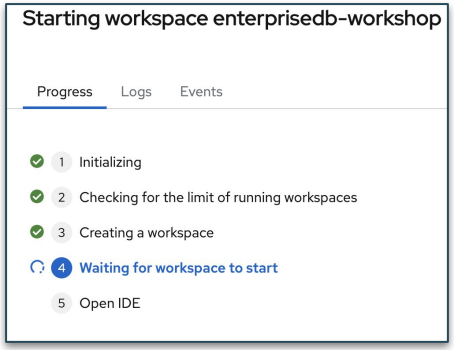
Press “Allow selected permissions”



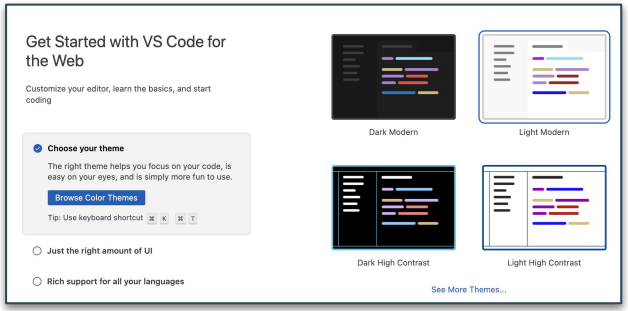
In the Select a Sample section search for “Workshop” and click on the tile

©EDB 2025 — ALL RIGHTS RESERVED.

Call to action: Open the DevSpace



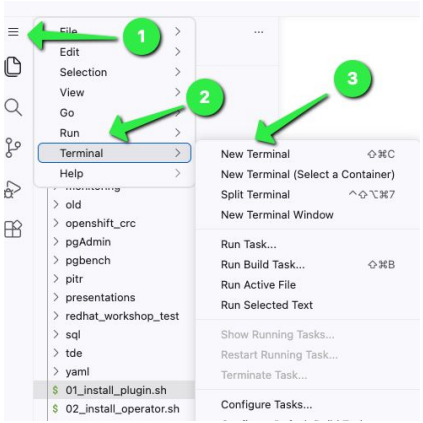
Your workshop is loading ...



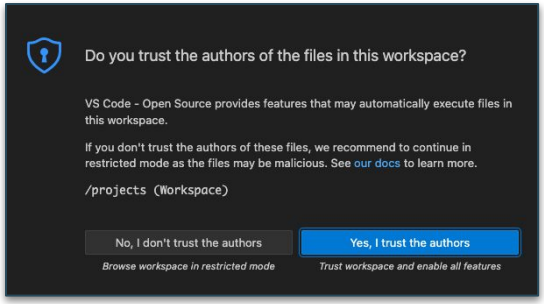
Select your theme

©EDB 2025 — ALL RIGHTS RESERVED.

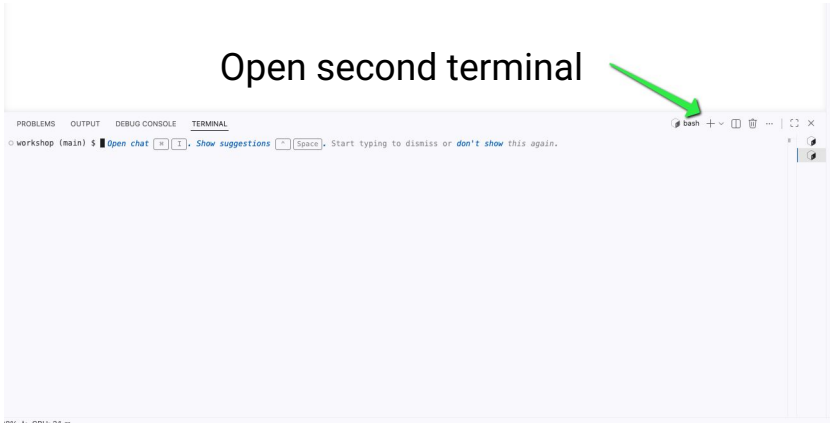
Call to action: Open two terminal window



Open a new terminal window (SHIFT + CTRL/CMD + C) and trust the authors

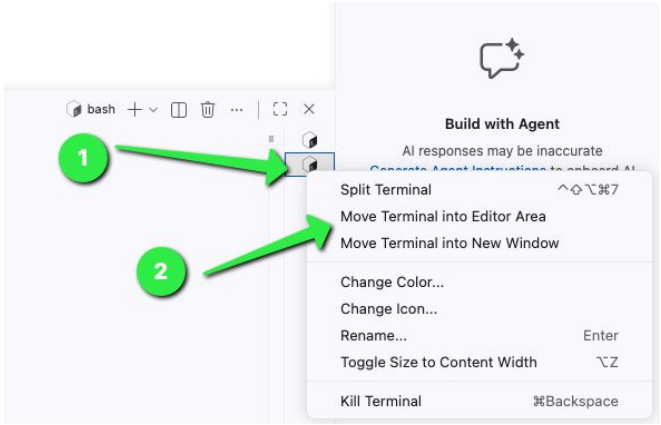


Open second terminal



©EDB 2025 — ALL RIGHTS RESERVED.

Call to action: Open two terminal window



Move second terminal into Editor Area (above)



Toggle “Chat” window off

©EDB 2025 — ALL RIGHTS RESERVED.

Two terminals are opened:

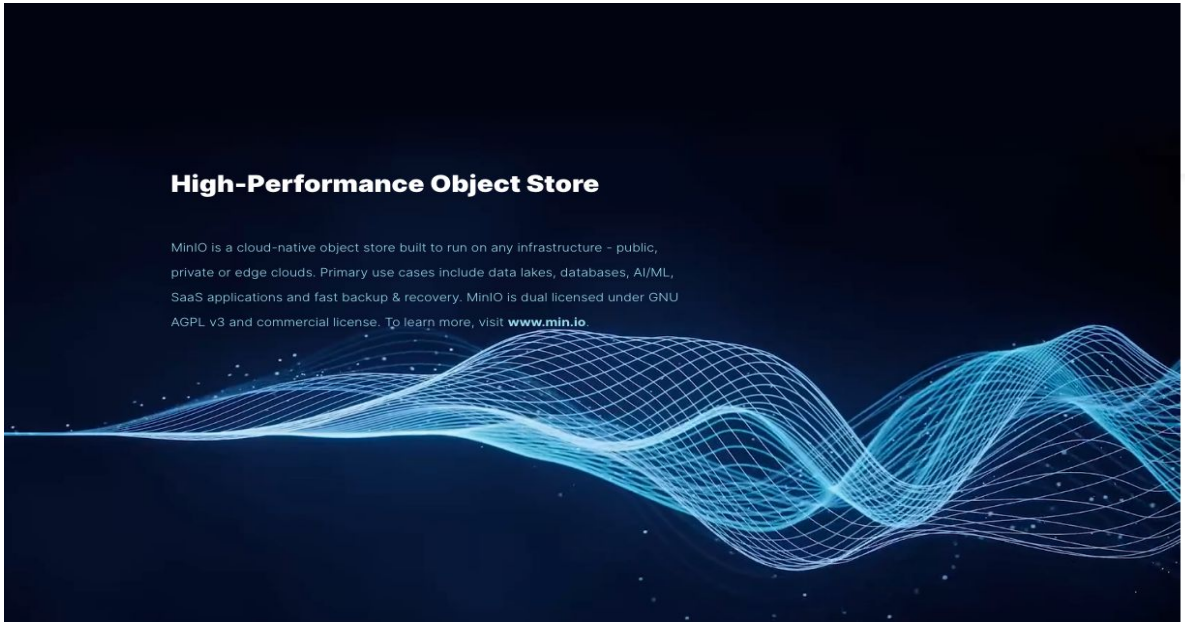


©EDB 2025 — ALL RIGHTS RESERVED.

MinIO Login

Got to the MinIO URL and enter username and password (minio/edb-workshop)

https://minio-ui-minio.apps.ocp.2lt49.sandbox2071.opentlc.com



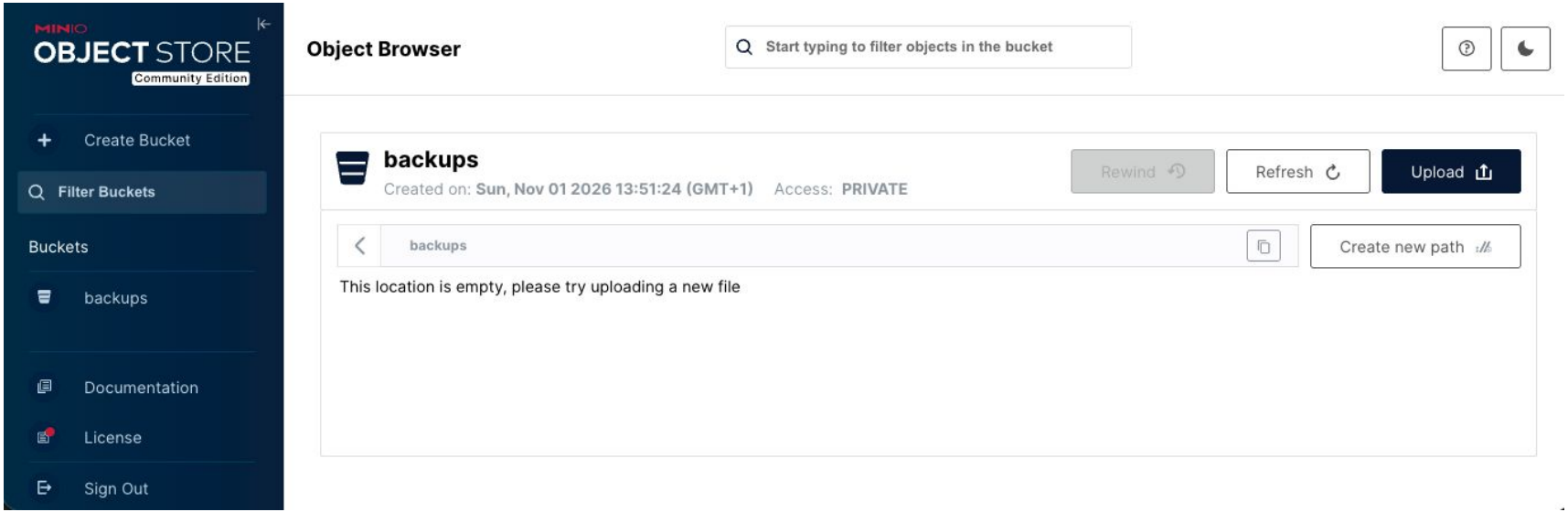


©EDB 2025 — ALL RIGHTS RESERVED.

MinIO First Logon

After your first logon the following screen will appear: check that the bucket “backups” is empty.

https://minio-ui-minio.apps.ocp.2lt49.sandbox2071.opentlc.com



©EDB 2025 — ALL RIGHTS RESERVED.

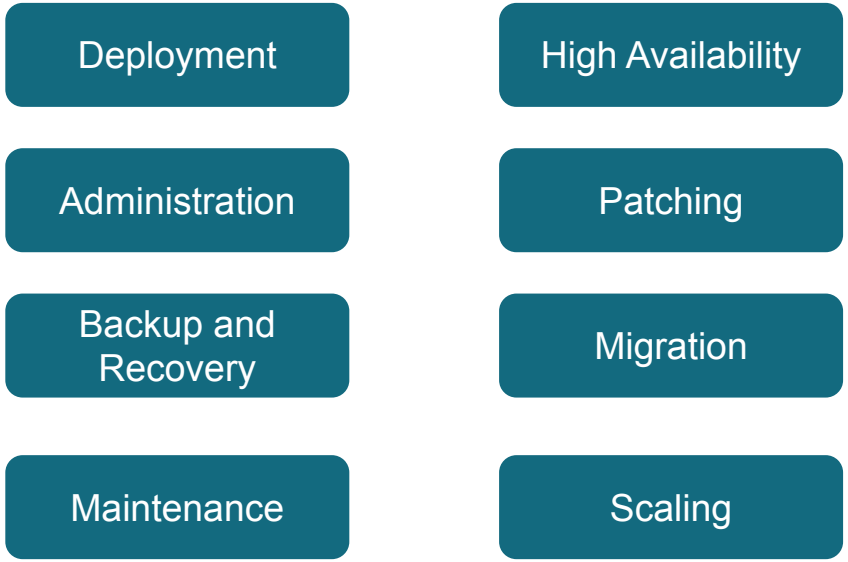
Use case
The environment



©EDB 2025 — ALL RIGHTS RESERVED.

Features shown during the demo

- Kubernetes plugin install
- Check the CloudNativePG operator status
- Postgres cluster install
- Insert data in the cluster
- Failover
- Backup
- Recovery
- Scale out/down
- Fencing
- Hibernation
- Rolling updates (minor and major)
- Migration



Last CloudNativePG tested version is 1.30.1

©EDB 2025 – ALL RIGHTS RESERVED.

This demo is in

https://github.com/borysneselovskyi/cnpg_openshift_workshop_ffm

https://tinyurl.com/cnpgopenshift

©EDB 2024 — ALL RIGHTS RESERVED.

Hands-on



©EDB 2025 — ALL RIGHTS RESERVED.

Plugging and operator installation

Scripts

```
./01_install_plugin.sh
# 02_install_operator.sh → already pre-installed ⚠️
```

Install plugin

CloudNativePG provides a plugin for kubectl to manage a Postgres cluster in Kubernetes. You can manage status, promote, certificates, restart, reload, maintenance, report, logs, destroy, hibernation, backups, …

Doc link

Install operator

CloudNativePG (CNPG) is an open-source operator designed to manage PostgreSQL workloads on any supported Kubernetes cluster. It fosters cloud-neutrality through seamless deployment in private, public, hybrid, and multi-cloud environments via its distributed topology feature.

Doc link

Code

```
# Install plugin

curl -sSfL \
  https://github.com/EnterpriseDB/kubectl-cnp/raw/main/install.sh
| sudo sh -s -- -b /usr/local/bin

# Check version

kubectl cnp version

# Install operator

version1=`oc cnp version | awk '{ print $2 }' | awk -F":" '{
print $2}'`
version2=${version1%??}

oc get packagemanifests -n openshift-marketplace
cloud-native-postgresql

oc describe packagemanifests -n openshift-marketplace
cloud-native-postgresql

oc apply -f ./yaml/operator-subscription.yaml
```

©EDB 2025 — ALL RIGHTS RESERVED.

Checking operator installation

Scripts

```
./03_check_operator_installed.sh
```

Checking operator installation

In Kubernetes, the operator is by default installed in the postgresql-operator-system namespace as a Kubernetes Deployment. The name of this deployment depends on the installation method. When installed through the manifest or the cnp plugin, by default, it is called postgresql-operator-controller-manager. When installed via Helm, by default, the deployment name is derived from the helm release name, appended with the suffix -edb-postgres-for-kubernetes (e.g., <name>-edb-postgres-for-kubernetes).

Doc link

Code

```
# Check operator installation

oc get deploy -A | grep postgres
NAMESPACE:      openshift-operators
NAME:           postgresql-operator-controller-manager
READY:          1/1
UP-TO-DATE:     1
AVAILABLE:      1
AGE:            4m
```

©EDB 2025 — ALL RIGHTS RESERVED.

Install Postgres cluster

Scripts

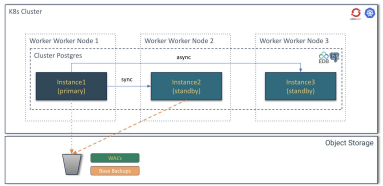
```
./04-prepare-cluster.sh
# Expected during first deletion: secrets "minio-creds" not found
./show_yaml.sh yaml/cluster-sample.yaml
./05_install_cluster.sh
./06_show_status.sh # in second terminal window
./07_insert_data.sh
```

Prepare Postgres cluster template

Setup credentials (AWS or Minio) and prepare context for your users

Deploy Postgres cluster in HA

Different types of architectures can be deployed. In our demo, 3 instances are deployed with HA available.



Insert Data

Insert data in `test` table

Doc link

Code

```
export kubectl_cmd="oc"
export cluster_name="cluster-example"
export TMP=$TMPDIR

# Get template
envsubst < templates/${cluster_name}-template.yaml >
$TMP/${cluster_name}.yaml

# Deploy Postgres cluster
${kubectl_cmd} apply -f $TMP/${cluster_name}.yaml
---------------------------------------------------------------------
ApiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: cluster-example
spec:
 instances: 3
 imageName: "docker.enterprisedb.com/k8s/postgresql:17.6"
 enableSuperuserAccess: true

 replicationSlots:
   highAvailability:
     enabled: true
```

©EDB 2025 — ALL RIGHTS RESERVED.

Monitoring

Show status (example - exact details will differ)

```
Cluster Summary
Name                 default/cluster-example
System ID:           7566239054312464401
PostgreSQL Image:    docker.enterprisedb.com/k8s/postgresql:17.6
Primary instance:    cluster-example-1
Primary promotion time:  2025-10-28 11:52:15 +0000 UTC (4m30s)
Status:              Cluster in healthy state
Instances:           3
Ready instances:     3
Size:                127M
Current Write LSN:   0/6032300 (Timeline: 1 - WAL File: 000000010000000000000006)

Continuous Backup status (Barman Cloud Plugin)
ObjectStore / Server name:      object-store/cluster-example
First Point of Recoverability:  -
Last Successful Backup:         -
Last Failed Backup:             -
Working WAL archiving:          OK
WALs waiting to be archived:    0
Last Archived WAL:              000000010000000000000005.00000028.backup   @   2025-10-28T11:54:11.908195Z
Last Failed WAL:                -

Streaming Replication status
Replication Slots Enabled
Name               Sent LSN   Write LSN  Flush LSN  Replay LSN  Write Lag  Flush Lag  Replay Lag  State      Sync State  Sync Priority  Replication Slot
----               --------   ---------  ---------  ----------  ---------  ---------  ----------  -----      ----------  -------------  ----------------
cluster-example-2  0/9000000  0/9000000  0/9000000  0/9000000   00:00:00   00:00:00   00:00:00    streaming  quorum      1              active
cluster-example-3  0/9000000  0/9000000  0/9000000  0/9000000   00:00:00   00:00:00   00:00:00    streaming  quorum      1              active

Instances status
Name               Current LSN  Replication role  Status  QoS         Manager Version  Node
----               -----------  ----------------  ------  ---         ---------------  ----
cluster-example-1  0/9000000    Primary           OK      Guaranteed  1.30.1           ocp4-multiarch-worker1-ppc64
cluster-example-2  0/9000000    Standby (sync)    OK      Guaranteed  1.30.1           ocp4-multiarch-w5gg4-master-2
cluster-example-3  0/9000000    Standby (sync)    OK      Guaranteed  1.30.1           ocp4-multiarch-w5gg4-master-1
```

©EDB 2025 — ALL RIGHTS RESERVED.

Promote and Failover

Scripts

```
./08_promote.sh
```

Promote

The meaning of this command is to promote a pod in the cluster to primary, so you can start with maintenance work or test a switch-over situation in your cluster:

Doc link

Code

```
# Promote

. ./config.sh

oc cnp promote cluster-example cluster-example-2
```

©EDB 2025 — ALL RIGHTS RESERVED.

Minor upgrade

Scripts

```
./09_upgrade.sh
```

Minor upgrade

To upgrade to a newer minor version, simply update the PostgreSQL container image reference in your cluster definition, either directly or via image catalogs. CloudNativePG will trigger a rolling update of the cluster, replacing each instance one by one, starting with the replicas. Once all replicas have been updated, it will perform either a switchover or a restart of the primary to complete the process.

Doc link

Code

```
# Minor upgrade

# From
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: cluster-example
spec:
 instances: 3
 imageName: "docker.enterprisedb.com/k8s/postgresql:17.6"
...

# To
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: cluster-example
spec:
 instances: 3
 imageName: "docker.enterprisedb.com/k8s/postgresql:17.7"
...
```

©EDB 2025 — ALL RIGHTS RESERVED.

Backups

Scripts

```
./show_yaml.sh yaml/backup.yaml
# watch minio console: before you only see the WALs created
./10_backup_cluster.sh
# subfolder “base” for the backup is created, verify “Backup Id”:
./11_backup_describe.sh
```

Backup

Backups are done using Barman with the new Barman Cloud Plugin. In cases where the plugin is not yet compatible (like in Red Hat OpenShift), we continue using Barman integrated in the Operator (without the “plugin”).

PostgreSQL natively provides first class backup and recovery capabilities based on file system level (physical) copy. These have been successfully used for more than 15 years in mission critical production databases, helping organizations all over the world achieve their disaster recovery goals with Postgres.

Doc link

Code

```
cat $TMP/backup.yaml

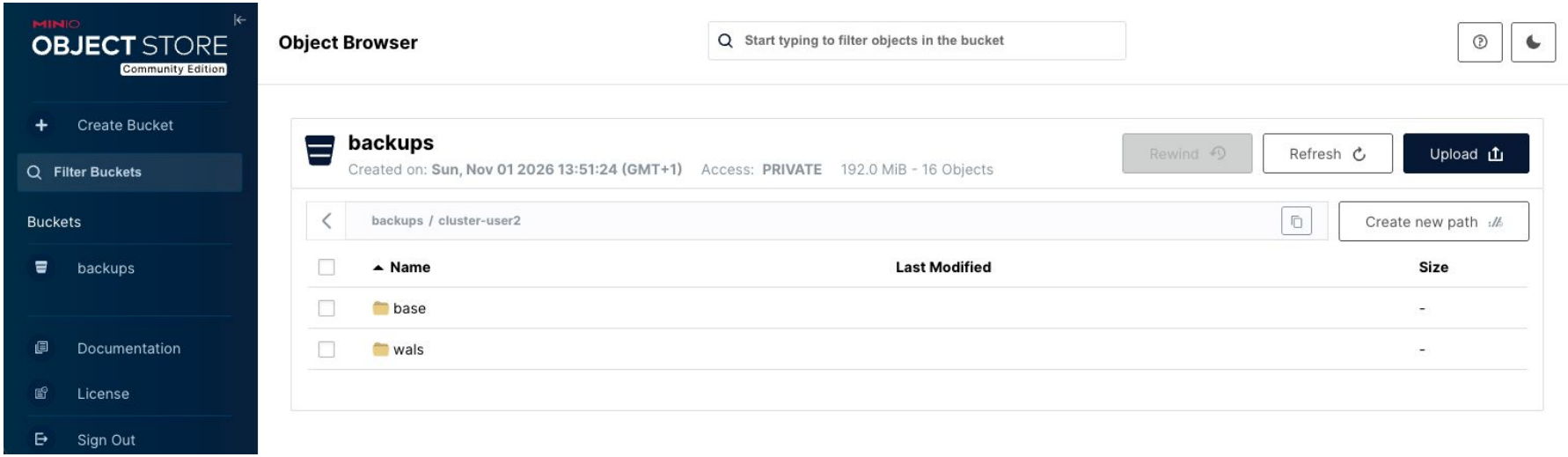
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Backup
metadata:
 name: cluster-example-backup-test
spec:
 cluster:
   name: cluster-example

oc apply -f ./yaml/backup.yaml
```

©EDB 2025 — ALL RIGHTS RESERVED.

Check Backup in MinIO UI

- Connect as user **minio** with the password: **edb-workshop**
- The page will appear:



©EDB 2025 — ALL RIGHTS RESERVED.

Recovery

Scripts

```
./show_yaml.sh yaml/restore.yaml
./12_restore_cluster.sh
oc get pods -w
oc get clusters
./13_check_restore.sh
oc delete cluster cluster-restore-user[N]  # don't forget! ⚠️
```

Recovery

In PostgreSQL, recovery refers to the process of starting an instance from an existing physical backup. PostgreSQL's recovery system is robust and feature-rich, supporting Point-In-Time Recovery (PITR)—the ability to restore a cluster to any specific moment, from the earliest available backup to the latest archived WAL file.

Doc link

Code

```
cat $TMP/restore.yaml

apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: cluster-restore
spec:
 instances: 1
 imageName: docker.enterprisedb.com/k8s/postgresql:17.7
 imagePullPolicy: IfNotPresent
 bootstrap:
   recovery:
     source: source
 plugins:
 - name: barman-cloud.cloudnative-pg.io
   isWALArchiver: true
   parameters:
     barmanObjectName: object-store
 externalClusters:
 - name: source
   plugin:
     name: barman-cloud.cloudnative-pg.io
     parameters:
       barmanObjectName: object-store
       serverName: cluster-example

oc apply -f $TMP/restore.yaml
```

©EDB 2025 — ALL RIGHTS RESERVED.

Promote and Failover

Scripts

```
./14_failover.sh
```

Automated Failover

In the case of unexpected errors on the primary for longer than the `.spec.failoverDelay` (by default 0 seconds), the cluster will go into failover mode. This may happen, for example, when:

- The primary pod has a disk failure
- The primary pod is deleted
- The `postgres` container on the primary has any kind of sustained failure

In the failover scenario, the primary cannot be assumed to be working properly.
Doc link

Code

```
# Automatic Failover (simulate failover killing pods of primary)
oc delete pvc/cluster-example \
          pvc/cluster-example-wal \
          pod/cluster-example \
          --force
```

©EDB 2025 — ALL RIGHTS RESERVED.

Scale out, scale in

Scripts

```
./15_scale.sh out
./15_scale.sh in
```

Scale out and in

The operator allows you to scale up and down the number of instances in a PostgreSQL cluster. New replicas are started up from the primary server and participate in the cluster's HA infrastructure.

Doc link

Code

```
# Scale out
oc scale cluster cluster-example --replicas=4


# Scale down
oc scale cluster cluster-example --replicas=2
```

©EDB 2025 — ALL RIGHTS RESERVED.

Fencing and Hibernation

Scripts

```
./22_fencing.sh on
./22_fencing.sh off
```

Fencing

Fencing is the process of protecting the data in one, more, or even all instances of a PostgreSQL cluster when they appear to be malfunctioning. When an instance is fenced, the PostgreSQL server process is guaranteed to be shut down, while the pod is kept running. This ensures that, until the fence is lifted, data on the pod isn't modified by PostgreSQL and that you can investigate file system for debugging and troubleshooting purposes.

Doc link

Code

```
replica=$(oc get pod \
-o=jsonpath="{range.items[*]}{.metadata.name}{'\t'}{.status.podIP}
{'\t'}{.metadata.labels.role}{'\n'}" | \
grep ${cluster_name}- | grep replica | awk '{print $1}' | head -1)

# Fencing on
oc cnp fencing on cluster-example ${replica}

# Fencing off
oc cnp fencing off cluster-example ${replica}
```

©EDB 2025 — ALL RIGHTS RESERVED.

Fencing and Hibernation

Scripts

```
./23_hibernation.sh on
./23_hibernation.sh off
```

Hibernation

CloudNativePG supports hibernation of a running PostgreSQL cluster in a declarative manner, through the cnpg.io/hibernation annotation. Hibernation enables saving CPU power by removing the database pods while keeping the database PVCs. This feature simulates scaling to 0 instances without losing their data.

Doc link

Code

```
# Hibernation on
oc annotate cluster cluster_example \
            --overwrite k8s.enterprisedb.io/hibernation=on

# Hibernation off
oc annotate cluster cluster_example \
            --overwrite k8s.enterprisedb.io/hibernation=off
```

©EDB 2025 — ALL RIGHTS RESERVED.

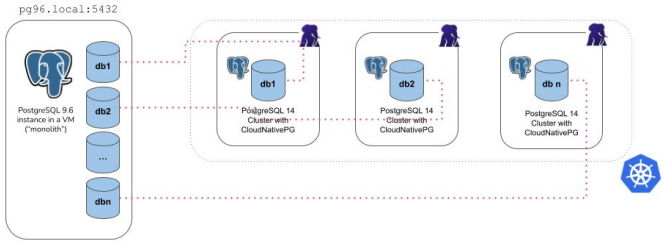
Out-of-place Major Upgrade (online data migration)

Scripts

```
./show_yaml.sh yaml/cluster-sample-major-upgrade-by-copy.yaml
./20_major_upgrade_by_copy.sh
oc get pods -w
./21_verify_data_migrated.sh
oc delete cluster cluster-user[N]-major
```

Major upgrade (with native logical replication)

- In this step we will migrate the app database from our existing cluster to the new cluster
- We will create the yaml file with the setting "**import**" in the bootstrap section
- The operator uses internally postgres tools pg_dump and pg_restore
- This method can be used to **migrate** another database or to run **out-of-the-place upgrade**
- Possible settings:
  - Microservices
  - Monolith



Doc link

Code

```
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: cluster-example-major
spec:
 instances: 1
 imageName: docker.enterprisedb.com/k8s/postgresql:18.1
 imagePullSecrets:
 - name: postgresql-operator-pull-secret
 enableSuperuserAccess: true

 bootstrap:
   initdb:
     import:
       type: microservice
       databases:
         - postgres
       source:
         externalCluster: cluster-example

 externalClusters:
   - name: cluster-example
     connectionParameters:
       host: cluster-example-rw
       user: postgres
       dbname: postgres
     password:
       name: cluster-example-superuser
       key: password
...
```

©EDB 2025 — ALL RIGHTS RESERVED.

Major upgrade (offline in-place)

Scripts

```
./24_major_upgrade_in_place.sh
# also watch: ./06_show_status.sh # in second terminal window
oc get pods -w
```

Major upgrade (offline in-place)

Major PostgreSQL releases introduce changes to the internal data storage format, requiring a more structured upgrade process.

CloudNativePG supports three methods for performing major upgrades:

- Logical dump/restore – Blue/green deployment, offline.
- Native logical replication – Blue/green deployment, online.
- **Physical with pg_upgrade** – In-place upgrade, offline (covered in the "Offline In-Place Major Upgrades" section below).

Doc link

Code

```
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: ${cluster_major_upgrade}
spec:
 instances: 1
 imageName: ${postgres_major_upgrade_image}

 storage:
   size: 2Gi
```

©EDB 2025 — ALL RIGHTS RESERVED.

Thank you

For any queries, contact:
borys.neselovskyi@enterprisedb.com /
christian.ballweg@enterprisedb.com

